FE 515: Lecture 2 Walkthrough

Using R to Compare Investments

Sean O'Leary

September 10, 2026

Contents

1 Lecture Goal **2**

2 Importing the Data **2**

2.1 Problem Description 2

3 Working With Dates **4**

3.1 Problem Description 4

4 From Prices to Returns **4**

4.1 Problem Description 4

4.2 Translating the Formula Into R 5

5 Constructing a 60/40 Portfolio **6**

5.1 Problem Description 6

6 Turning Returns Into Wealth **8**

6.1 Problem Description 8

7 Visualizing the Investment **10**

7.1 Problem Description 10

8 Measuring Drawdown **11**

8.1 Problem Description 11

8.2 Finding the Running Peak in R 12

9 Comparing All Three Investments **13**

9.1 Problem Description 13

9.2 Building a Reusable Summary Function 14

1 Lecture Goal

In the first lecture, we introduced enough R syntax to store data, work with vectors and data frames, use functions, and calculate returns.

Today, we are going to solve a realistic problem.

Suppose you are a junior analyst and are given historical price data for two investments:

- an equity investment,
- a bond investment.

An investor has **$10,000** and wants to know what would have happened if they had invested the money in:

1. the equity investment,
2. the bond investment,
3. a 60/40 portfolio containing 60% equity and 40% bonds.

Our job is to use the data to answer:

> **Which investment produced the strongest performance, and what risk did the investor experience along the way?**

The workflow for today's analysis will be:

$$\text{Raw Data} \rightarrow \text{Inspect} \rightarrow \text{Returns} \rightarrow \text{Portfolio} \rightarrow \text{Wealth} \rightarrow \text{Risk} \rightarrow \text{Decision}$$

2 Importing the Data

2.1 Problem Description

Before doing any financial calculation, we need to understand the data we were given.

The file `FE515_L2_Investment_Data.csv` contains three variables:

- `Date`
- `Equity`
- `Bond`

The equity and bond columns contain price levels.

A common mistake in financial analysis is to immediately begin calculations without first inspecting the dataset. Before using the data, we should ask:

- Did R read the file correctly?
- How many observations do we have?
- What type of object is each column?
- Are there missing values?

- Do the prices appear reasonable?

We begin by importing the file.

```
data <- read.csv("FE515_L2_Investment_Data.csv")
```

Now inspect the first few observations.

```
head(data)
```

```
##         Date  Equity     Bond
## 1 2025-01-02 99.4386 100.0493
## 2 2025-01-03 97.7574 100.5151
## 3 2025-01-06 96.4289 101.1203
## 4 2025-01-07 95.1308 101.5123
## 5 2025-01-08 96.6198 101.7740
## 6 2025-01-09 94.8651 101.4679
```

We can also inspect the structure of the dataset.

```
str(data)
```

```
## 'data.frame':    260 obs. of  3 variables:
##  $ Date  : chr  "2025-01-02" "2025-01-03" "2025-01-06" "2025-01-07" ...
##  $ Equity: num  99.4 97.8 96.4 95.1 96.6 ...
##  $ Bond  : num  100 101 101 102 102 ...
```

And check the dimensions.

```
dim(data)
```

```
## [1] 260   3
```

The `dim()` function returns:

$$(\text{number of rows},\ \text{number of columns})$$

Finally, check whether any observations are missing.

```
colSums(is.na(data))
```

```
##   Date Equity   Bond
##      0      0      0
```

2.1.1 Quick Check

Before moving forward:

1. What does each row represent?
2. What does each column represent?
3. Does R currently recognize `Date` as a date?
4. Why is checking for missing values important before computing returns?

3 Working With Dates

3.1 Problem Description

The `Date` column was imported from a CSV file.

CSV files store text and numbers, but they do not automatically preserve special R object types. Therefore, R may initially treat the dates as character strings.

For financial time series, we want R to understand that these observations occur in chronological order.

We can convert the column using:

```
data$Date <- as.Date(data$Date)
```

Check the structure again.

```
str(data)
```

```
## 'data.frame':    260 obs. of  3 variables:
##  $ Date  : Date, format: "2025-01-02" "2025-01-03" ...
##  $ Equity: num  99.4 97.8 96.4 95.1 96.6 ...
##  $ Bond  : num  100 101 101 102 102 ...
```

Now R recognizes each observation as an actual calendar date.

This matters because dates can now be used naturally for:

- plotting,
- sorting,
- filtering,
- identifying particular market periods.

For example:

```
min(data$Date)
```

```
## [1] "2025-01-02"
```

```
max(data$Date)
```

```
## [1] "2025-12-31"
```

4 From Prices to Returns

4.1 Problem Description

Price levels tell us how much an asset costs, but they do not directly tell us how well the investment performed.

If an asset moves from price P_{t-1} to price P_t, its **simple return** is

$$r_t = \frac{P_t - P_{t-1}}{P_{t-1}}.$$

This can also be written as

$$r_t = \frac{P_t}{P_{t-1}} - 1.$$

For example, if a stock rises from \$100 to \$103,

$$r_t = \frac{103}{100} - 1 = 0.03.$$

The investor earned a 3% return.

In Lecture 1, we also worked with continuously compounded, or log, returns:

$$r_t^{\log} = \log(P_t) - \log(P_{t-1}).$$

Both are useful. Today we will use **simple returns** because they have a particularly convenient interpretation when constructing portfolios and tracking investment value.

4.2 Translating the Formula Into R

Suppose we have the price vector

$$P_1, P_2, \ldots, P_n.$$

To compute returns, we need to compare

$$P_2, \ldots, P_n$$

with

$$P_1, \ldots, P_{n-1}.$$

In R, we can create those two vectors by removing the first or last observation.

```
n <- nrow(data)

data$Equity[-1][1:5]
```

```
## [1] 97.7574 96.4289 95.1308 96.6198 94.8651
```

```
data$Equity[-n][1:5]
```

```
## [1] 99.4386 97.7574 96.4289 95.1308 96.6198
```

Now implement the return formula directly.

```r
equity_return <- data$Equity[-1] / data$Equity[-n] - 1
bond_return   <- data$Bond[-1]   / data$Bond[-n]   - 1
```

Inspect the first several returns.

```r
head(equity_return)
```

```
## [1] -0.01690692 -0.01358976 -0.01346173  0.01565213 -0.01816087 -0.02698885
```

```r
head(bond_return)
```

```
## [1]  0.004655705  0.006020986  0.003876571  0.002578013 -0.003007644
## [6] -0.002757522
```

Because a return requires **two prices**, n prices produce only $n - 1$ returns.

```r
length(data$Equity)
```

```
## [1] 260
```

```r
length(equity_return)
```

```
## [1] 259
```

The first return corresponds to the second date in the original dataset.

```r
return_dates <- data$Date[-1]
head(return_dates)
```

```
## [1] "2025-01-03" "2025-01-06" "2025-01-07" "2025-01-08" "2025-01-09"
## [6] "2025-01-10"
```

4.2.1 Quick Check

If there are 250 price observations, how many daily returns should there be?

Why?

5 Constructing a 60/40 Portfolio

5.1 Problem Description

The investor does not necessarily need to choose only one asset.

Suppose the portfolio places:

$$w_E = 0.60$$

in equity and

$$w_B = 0.40$$

in bonds.

For a two-asset portfolio, the one-period portfolio return is

$$r_{p,t} = w_E r_{E,t} + w_B r_{B,t}.$$

More generally,

$$r_{p,t} = \sum_{i=1}^{N} w_i r_{i,t},$$

where the portfolio weights satisfy

$$\sum_{i=1}^{N} w_i = 1.$$

For our 60/40 portfolio,

$$r_{p,t} = 0.60 r_{E,t} + 0.40 r_{B,t}.$$

This tells us something useful about R: once returns are stored as vectors, the mathematical portfolio formula can be translated almost directly into code.

```
portfolio_return <- 0.60 * equity_return + 0.40 * bond_return
```

Inspect the first few portfolio returns.

```
head(portfolio_return)
```

```
## [1] -0.008281867 -0.005745464 -0.006526411  0.010422485 -0.012099582
## [6] -0.017296319
```

To verify the calculation manually, compare the first portfolio return with the formula.

```
0.60 * equity_return[1] + 0.40 * bond_return[1]
```

```
## [1] -0.008281867
```

```
portfolio_return[1]
```

```
## [1] -0.008281867
```

These should be identical.

Important assumption: this calculation treats the portfolio as being returned to 60% equity and 40% bonds each period. In other words, it represents a regularly rebalanced 60/40 strategy. We will study portfolio mechanics in more detail later in the course.

5.1.1 Quick Check

Suppose on one day:

$$r_E = 2\%, \qquad r_B = -0.5\%.$$

Without using R, what should the 60/40 portfolio return be?

6 Turning Returns Into Wealth

6.1 Problem Description

Returns are useful, but the investor's question was not:

> "What vector of returns would I have earned?"

The investor wants to know:

> **What happened to my $10,000?**

Suppose the initial investment is

$$V_0 = 10{,}000.$$

After one period,

$$V_1 = V_0(1 + r_1).$$

After two periods,

$$V_2 = V_1(1 + r_2) = V_0(1 + r_1)(1 + r_2).$$

After T periods,

$$V_T = V_0 \prod_{t=1}^{T} (1 + r_t).$$

This is the effect of compounding.
If returns are

$$r_1, r_2, \ldots, r_T,$$

we therefore need the sequence

$$(1 + r_1),\ (1 + r_1)(1 + r_2),\ \ldots,\ \prod_{t=1}^{T} (1 + r_t).$$

R has a function designed for cumulative products:

```
cumprod()
```

For example:

```
example_returns <- c(0.10, -0.05, 0.02)
cumprod(1 + example_returns)
```

```
## [1] 1.1000 1.0450 1.0659
```

Now apply the same idea to our investments.

```
initial_investment <- 10000

equity_wealth <- c(
  initial_investment,
  initial_investment * cumprod(1 + equity_return)
)

bond_wealth <- c(
  initial_investment,
  initial_investment * cumprod(1 + bond_return)
)

portfolio_wealth <- c(
  initial_investment,
  initial_investment * cumprod(1 + portfolio_return)
)
```

We include the initial $10,000 as the first observation so the wealth vectors have the same number of observations as our dates.

```
length(data$Date)
```

```
## [1] 260
```

```
length(equity_wealth)
```

```
## [1] 260
```

Now look at the ending values.

```
tail(equity_wealth, 1)
```

```
## [1] 7530.567
```

```
tail(bond_wealth, 1)
```

```
## [1] 10216.96
```

```r
tail(portfolio_wealth, 1)
```

```
## [1] 8558.383
```

These values answer our first major financial question:

> If we invested $10,000 at the beginning of the sample, how much would each strategy be worth at the end?

7 Visualizing the Investment

7.1 Problem Description

A final account value tells us where an investment ended.

It does **not** tell us what happened along the way.

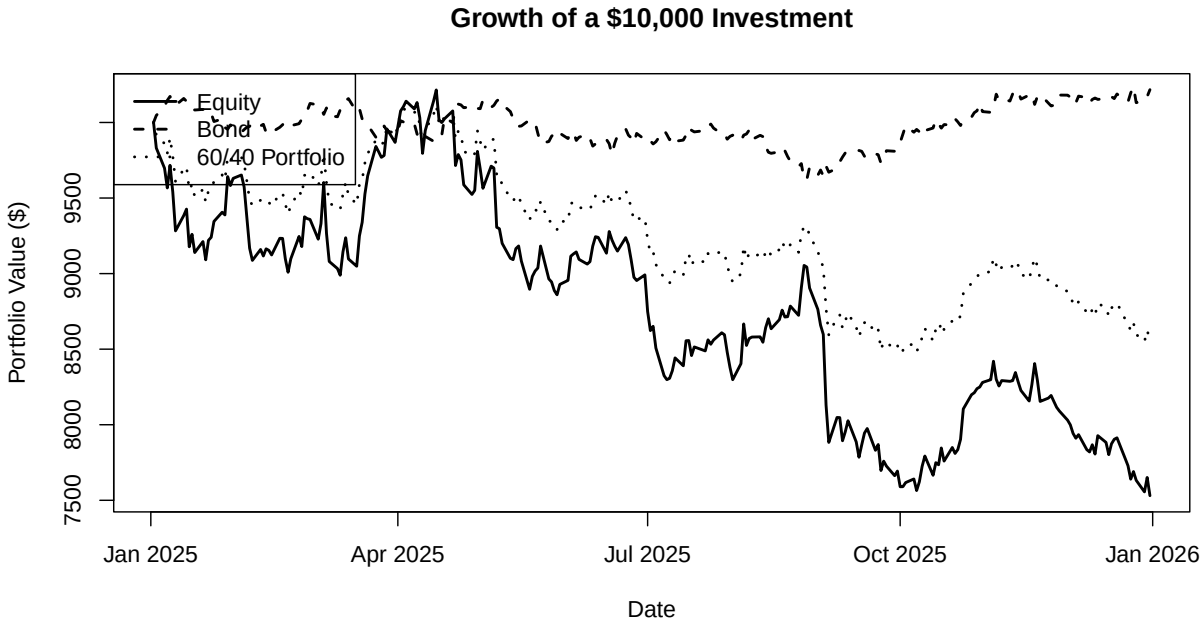
Two investments could both finish at $11,000 while producing completely different experiences for the investor.

We therefore want to see the entire wealth path.

```r
plot(
  data$Date,
  equity_wealth,
  type = "l",
  lwd = 2,
  xlab = "Date",
  ylab = "Portfolio Value ($)",
  main = "Growth of a $10,000 Investment"
)

lines(data$Date, bond_wealth, lwd = 2, lty = 2)
lines(data$Date, portfolio_wealth, lwd = 2, lty = 3)

legend(
  "topleft",
  legend = c("Equity", "Bond", "60/40 Portfolio"),
  lty = c(1, 2, 3),
  lwd = 2
)
```



The graph is doing more than making the analysis look attractive. It is helping us identify financial behavior that is difficult to see from a single number.

8 Measuring Drawdown

8.1 Problem Description

Volatility is one way to describe risk, but investors often think about losses relative to their previous high.

Suppose an investment reaches $12,000 and later falls to $10,800.

Relative to its previous peak, the investment has fallen

$$\frac{10{,}800}{12{,}000} - 1 = -0.10.$$

This is a 10% **drawdown**.

At each time t, define the running peak as

$$M_t = \max_{s \le t} V_s.$$

The drawdown is then

$$D_t = \frac{V_t}{M_t} - 1.$$

A drawdown is zero whenever the investment is at a new high. It becomes negative when the investment falls below its previous peak.

The **maximum drawdown** over the sample is

$$\text{MDD} = \min_t D_t.$$

Because drawdowns are negative, the minimum value represents the largest peak-to-trough loss.

8.2 Finding the Running Peak in R

We need, for every date, the largest wealth value observed **up to that date**.

R provides the function

```
cummax()
```

which calculates a cumulative maximum.

```
portfolio_peak <- cummax(portfolio_wealth)

head(portfolio_wealth)
```

```
## [1] 10000.000  9917.181  9860.203  9795.851  9897.948  9778.187
```

```
head(portfolio_peak)
```

```
## [1] 10000 10000 10000 10000 10000 10000
```

Now calculate drawdown directly from the formula.

```
portfolio_drawdown <- portfolio_wealth / portfolio_peak - 1
```

Find the maximum drawdown.

```
min(portfolio_drawdown)
```

```
## [1] -0.1621141
```

To express it as a percentage:

```
100 * min(portfolio_drawdown)
```

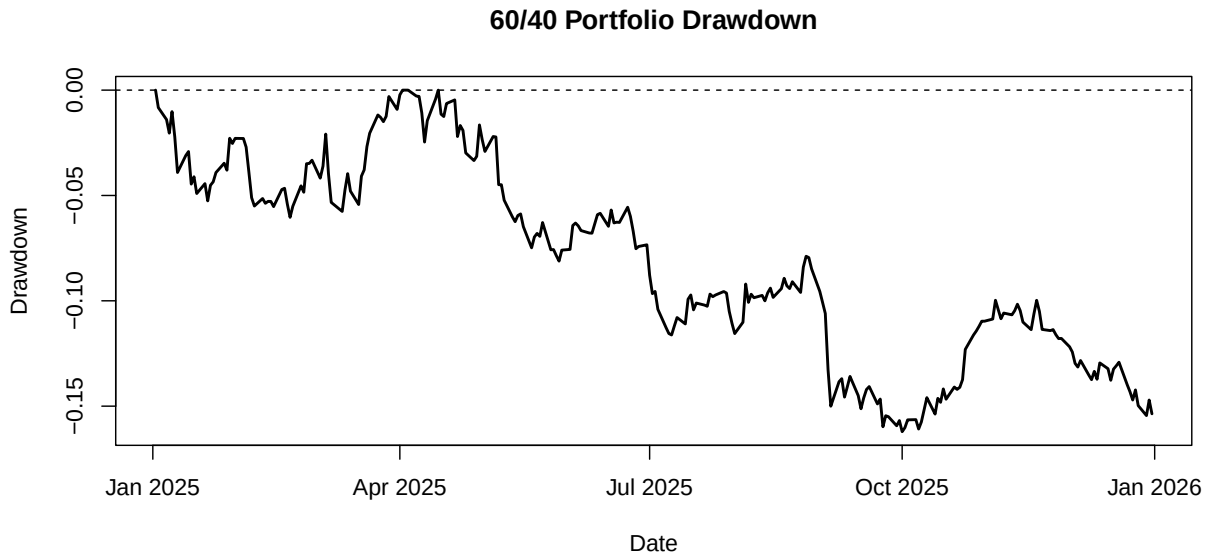
```
## [1] -16.21141
```

We can also visualize the drawdown through time.

```
plot(
  data$Date,
  portfolio_drawdown,
  type = "l",
  lwd = 2,
  xlab = "Date",
```

```
  ylab = "Drawdown",
  main = "60/40 Portfolio Drawdown"
)

abline(h = 0, lty = 2)
```



9 Comparing All Three Investments

9.1 Problem Description

We have now repeated several calculations for multiple investments.

For each strategy, we would like to know:

1. ending wealth,
2. total return,
3. average daily return,
4. daily volatility,
5. maximum drawdown.

The total return from initial wealth V_0 to final wealth V_T is

$$R_{\text{total}} = \frac{V_T}{V_0} - 1.$$

Daily volatility is measured using the sample standard deviation of daily returns:

$$\sigma_r = \sqrt{\frac{1}{n-1}\sum_{t=1}^{n}(r_t - \bar{r})^2}.$$

R already provides `mean()` and `sd()` for these calculations.

For maximum drawdown, we repeat the formula from the previous section:

$$\text{MDD} = \min_t \left(\frac{V_t}{\max_{s \le t} V_s} - 1 \right).$$

At this point, copying and rewriting the same code three times would be inefficient.

This is a natural reason to create our own function.

9.2 Building a Reusable Summary Function

```
investment_summary <- function(name, returns, wealth) {

  total_return <- tail(wealth, 1) / wealth[1] - 1
  average_return <- mean(returns)
  volatility <- sd(returns)
  max_drawdown <- min(wealth / cummax(wealth) - 1)

  data.frame(
    Investment = name,
    Ending.Wealth = tail(wealth, 1),
    Total.Return = total_return,
    Average.Daily.Return = average_return,
    Daily.Volatility = volatility,
    Maximum.Drawdown = max_drawdown
  )
}
```

The function accepts three pieces of information:

- a name,
- a vector of returns,
- a vector of wealth values.

It returns one row containing the statistics we want.

Now use the same tool for all three investments.

```
equity_summary <- investment_summary(
  "Equity",
  equity_return,
  equity_wealth
)

bond_summary <- investment_summary(
  "Bond",
  bond_return,
  bond_wealth
)
```

```r
portfolio_summary <- investment_summary(
  "60/40 Portfolio",
  portfolio_return,
  portfolio_wealth
)

summary_table <- rbind(
  equity_summary,
  bond_summary,
  portfolio_summary
)

summary_table
```

```
##        Investment Ending.Wealth Total.Return Average.Daily.Return
## 1          Equity      7530.567   -0.2469433        -1.014914e-03
## 2            Bond     10216.963    0.0216963         8.976444e-05
## 3 60/40 Portfolio      8558.383   -0.1441617        -5.730427e-04
##   Daily.Volatility Maximum.Drawdown
## 1      0.012605520       -0.2628127
## 2      0.003718444       -0.0541150
## 3      0.007467362       -0.1621141
```

For easier reading, we can round the numeric values.

```r
summary_table$Ending.Wealth <- round(summary_table$Ending.Wealth, 2)
summary_table$Total.Return <- round(100 * summary_table$Total.Return, 2)
summary_table$Average.Daily.Return <- round(100 * summary_table$Average.Daily.Return, 3)
summary_table$Daily.Volatility <- round(100 * summary_table$Daily.Volatility, 3)
summary_table$Maximum.Drawdown <- round(100 * summary_table$Maximum.Drawdown, 2)

names(summary_table) <- c(
  "Investment",
  "Ending Wealth ($)",
  "Total Return (%)",
  "Average Daily Return (%)",
  "Daily Volatility (%)",
  "Maximum Drawdown (%)"
)

summary_table
```

```
##        Investment Ending Wealth ($) Total Return (%) Average Daily Return (%)
## 1          Equity           7530.57           -24.69                    -0.101
## 2            Bond          10216.96             2.17                     0.009
## 3 60/40 Portfolio           8558.38           -14.42                    -0.057
##   Daily Volatility (%) Maximum Drawdown (%)
## 1                1.261               -26.28
## 2                0.372                -5.41
## 3                0.747               -16.21
```

10 Making the Investment Decision

We can now return to the original assignment.

The investor had $10,000 and asked us to compare:

- equity,
- bonds,
- a 60/40 portfolio.

We used R to move from raw price data to a financial comparison.

Our workflow was:

$$\text{Prices} \rightarrow \text{Returns} \rightarrow \text{Portfolio Returns} \rightarrow \text{Wealth} \rightarrow \text{Drawdown} \rightarrow \text{Comparison}$$

The important point is not memorizing every R function that appeared today.

Instead, notice how each function entered the analysis because we needed it to answer a financial question:

- `read.csv()` because we needed to load data,
- `str()` because we needed to understand the data,
- `as.Date()` because the observations occur through time,
- vector arithmetic because returns are mathematical transformations of prices,
- `cumprod()` because investment wealth compounds,
- `cummax()` because drawdown depends on the previous peak,
- `plot()` because a full investment path contains information that a final value does not,
- a user-defined function because we wanted to repeat the same analysis efficiently.